CONTEXT FREE GRAMMAR

1	<program>	→	<global_decl> <function_dec> <functions> Team() { <local_decl> <team_statements> }
2	<global_decl>	→	<global_declared> <global_decl>
3	<global_decl>	→	λ
4	<global_declared>	→	<variable_dec>
5	<global_declared>	→	<array_dec>
6	<global_declared>	→	<constant_dec>
7	<function_dec>	→	Wilt <function_decls>< <function_dec>
8	<function_dec>	→	λ
9	<functions>	→	<function_definition> <functions>
10	<functions>	→	λ
11	<variable_dec>	→	<variable_decls>
12	<variable_decls>	→	<data_type> identifier <initialize_end> <decls_end>;
13	<decls_end>	→	, identifier <initialize_end> <decls_end>
14	<decls_end>	→	λ
15	<data_type>	→	Feint
16	<data_type>	→	Floater
17	<data_type>	→	Chant
18	<data_type>	→	Stat
19	<initialize_end>	→	= <global_value> <decls_end>
20	<initialize_end>	→	λ
21	<array_dec>	→	<data_type> <array_decls>
22	<array_decls>	→	identifier[<array_size>] <2d> <array_initialization>;
23	<2d>	→	λ

24	<2d>	→	[<array_size>]
25	<array_initialization>	→	= { <row> <rows> }
26	<array_initialization>	→	λ
27	<row>	→	{<array_elem> <elements> }
28	<rows>	→	, {<array_elem> <elements> }
29	<rows>	→	λ
30	<element_index>	→	Feint literal
31	<array_size>	→	Feint literal
32	<array_elem>	→	Feint literal
33	<array_elem>	→	Floater literal
34	<array_elem>	→	Chant literal
35	<elements>	→	, <array_elem><elements>
36	<elements>	→	λ
37	<constant_dec>	→	Const <data_type> identifier = <literal>;
38	<function_decls>	→	Wilt <data_type> identifier(<parameters>) <function_definition>;
39	<parameters>	→	<parameter><parameters_end>
40	<parameters>	→	λ
41	<parameter>	→	<data_type> identifier
42	<parameters_end>	→	, <parameters>
43	<parameters_end>	→	λ
44	<function_definition>	→	λ
45	<function_definition>	→	{<team_statements> <rebound_statements>}
46	<function_definition>	→	Dunk identifier(<parameters>){<team_statements>}

47	<local_decl>	→	<local_declared> <local_decl>
48	<local_decl>	→	λ
49	<local_declared>	→	<variable_dec>
50	<local_declared>	→	<array_dec>
51	<rebound_statements>	→	Rebound <local_value>;
52	<rebound_statements>	→	λ
53	<team_statements>	→	<statement> <next_statements>
54	<team_statements>	→	<dribble_statements> <next_statements>
55	<team_statements>	→	<array_dec><next_statements>
56	<next_statements>	→	<team_statements> <next_statements>
57	<next_statements>	→	λ
58	<statement>	→	<announce_statement>
59	<statement>	→	<inbound_statement>
60	<statement>	→	<forward_statement>
61	<statement>	→	<play_statement>
62	<statement>	→	<run_play_statement>
63	<statement>	→	<variable_dec>
64	<statement>	→	<assignment_statement>
65	<statement>	→	<switch_statement>
66	<statement>	→	<array_manipulation>
67	<array_manipulation>	→	identifier[<element_index>] <2d_manipulation> <array_man>;
68	<2d_manipulation>	→	[<element_index>]
69	<2d_manipulation>	→	λ
70	<array_man>	→	= <array_elem>
71	<array_man>	→	λ

72	<switch_statement>	→	Switch (<switch_exp>) {<case_list> <default_case>}
73	<switch_exp>	→	identifier
74	<switch_exp>	→	Feint literal
75	<switch_exp>	→	Chant literal
76	<case_list>	→	<case> <case_list>
77	<case>	→	Get <switch_value>:
78	<switch_value>	→	‘arith_op’
79	<switch_value>	→	‘alphabet’
80	<switch_value>	→	Feint literal
81	<switch_value>	→	Floater literal
82	<default_case>	→	Default: <Announce>
83	<assignment_statement>	→	<data_type> identifier <assign_op> <local_value>;
84	<inbound_statement>	→	Inbound(<input> <inputs>);
85	<input>	→	Chant literal
86	<inputs>	→	, identifier
87	<announce_statement>	→	Announce(<outputs>);
88	<outputs>	→	<output><output_end>
89	<output_end>	→	& <outputs>
90	<output_end>	→	λ
91	<output>	→	<global_value>
92	<output>	→	<local_value>
93	<output>	→	<chant_concat>
94	<global_value>	→	identifier
95	<global_value>	→	<literal>
96	<global_value>	→	<expression>

97	<local_value>	→	Identifier <arr_identifier>
98	<arr_identifier>	→	[<element_index>]
99	<local_value>	→	<expression>
100	<local_value>	→	<func_call>
101	<local_value>	→	<type_casted>
102	<local_value>	→	<escape_sequence>
103	<escape_sequence>	→	\<esc_charac>
104	<esc_charac>	→	n
105	<esc_charac>	→	t
106	<esc_charac>	→	\
107	<esc_charac>	→	"
108	<esc_charac>	→	@
109	<esc_charac>	→	&
110	<literal>	→	Feint literal
111	<literal>	→	Floater literal
112	<literal>	→	Chant literal
113	<literal>	→	Stat literal
114	<type_casted>	→	<data_type> (<type_cast1>)
115	<type_cast1>	→	<literal>
116	<type_cast1>	→	identifier
117	<assign_op>	→	=
118	<assign_op>	→	+=
119	<assign_op>	→	-=
120	<assign_op>	→	*=
121	<assign_op>	→	%=

122	<assign_op>	→	/=
123	<arith_op>	→	+
124	<arith_op>	→	-
125	<arith_op>	→	*
126	<arith_op>	→	/
127	<arith_op>	→	%
128	<rel_op>	→	<
129	<rel_op>	→	>
130	<rel_op>	→	<=
131	<rel_op>	→	>=
132	<rel_op>	→	<relation_equality>
133	<relation_equality>	→	==
134	<relation_equality>	→	!=
135	<log_op>	→	AND
136	<log_op>	→	OR
137	<log_op>	→	NOT
138	<expression>	→	<arithmetic_expr>
139	<expression>	→	<relational_expr>
140	<expression>	→	<logical_expr>
141	<arithmetic_expr>	→	<term> <arithexpr_end>
142	<arithexpr_end>	→	<arith_op> <term> <arithexpr_end>
143	<arithexpr_end>	→	λ
144	<term>	→	identifier
145	<term>	→	<num_literal>
146	<term>	→	<num_type_casted>

147	<term>	→	(<arithmetic_expr>)
148	<num_literal>	→	Feint literal
149	<num_literal>	→	Floater literal
150	<relational_expr>	→	<rel_operand> <rel_op> <rel_operand> <rel_end>
151	<rel_end>	→	<rel_op> <rel_operand> <rel_end>
152	<rel_end>	→	λ
153	<rel_operand>	→	<arithmetic_expr>
154	<rel_operand>	→	identifier
155	<rel_operand>	→	<type_casted>
156	<rel_operand>	→	<num_literal>
157	<rel_operand>	→	(<rel_expr>)
158	<rel_expr>	→	<rel_operand_eq> <relation_equality> <rel_operand_eq> <rel_end_eq>
159	<rel_end_eq>	→	<relation_equality> <rel_operand_eq> <rel_end_eq>
160	<rel_end_eq>	→	λ
161	<rel_operand_eq>	→	Stat literal
162	<rel_operand_eq>	→	Chant literal
163	<rel_operand_eq>	→	identifier
164	<rel_operand_eq>	→	(<relational_expr>)
165	<logical_expr>	→	<log_operand><log_op><log_operand> <log_end>
166	<log_end>	→	<log_op><log_operand><log_end>
167	<log_end>	→	λ
168	<log_operand>	→	<logical_expr>
169	<log_operand>	→	<relational_expr>

170	<log_operand>	→	identifier
171	<log_operand>	→	Stat literal
172	<log_operand>	→	(<logical_expr>)
173	<chant_concat>	→	<chant_operand> <chant_concat_end>
174	<chant_concat>	→	<concat_op> <concat_end>
175	<chant_operand>	→	Chant literal
176	<chant_operand>	→	identifier
177	<chant_operand>	→	<chant_type_casted>
178	<chant_type_casted>	→	Chant (<chant_typcst>)
179	<chant_typcst>	→	<literal>
180	<chant_typcst>	→	identifier
181	<chant_concat_end>	→	& <concat_op>
182	<chant_concat_end>	→	λ
183	<concat_end>	→	& <chant_operand>
184	<concat_end>	→	λ
185	<concat_op>	→	<chant_operand>
186	<concat_op>	→	<num_literal>
187	<concat_op>	→	identifier
188	<concat_op>	→	<num_type_casted>
189	<num_type_casted>	→	<num_data_type>(num_literal)
190	<num_data_type>	→	Feint
191	<num_data_type>	→	Floater
192	<dribble_statements>	→	Dribble (<condition>) { <team_statements> } <pass_statements> <shoot_scp>
193	<pass_statements>	→	<pass_scp> <pass_statements>
194	<pass_statements>	→	λ

195	<pass_scp>	→	Pass (<condition>) { <team_statements> }
196	<shoot_scp>	→	Shoot { <team_statements> }
197	<shoot_scp>	→	λ
198	<condition>	→	<relational_expr>
199	<condition>	→	<logical_expr>
200	<condition>	→	Stat literal
201	<forward_statement>	→	Forward (<forward_initialization>; <forward_condition>; <forward_incdec>) {<loop_stmts> <next_loopstmts>}
202	<forward_initialization>	→	<numdata_type> identifier = <initial_operand>
203	<numdata_type>	→	Feint
204	<initial_operand>	→	<num_literal>
205	<initial_operand>	→	<arithmetic_expr>
206	<forward_condition>	→	<forward_operand><forward_op><forward_operand>
207	<forward_operand>	→	identifier
208	<forward_operand>	→	<arithmetic_expr>
209	<forward_operand>	→	<num_literal>
210	<forward_op>	→	<rel_op>
211	<forward_incdec>	→	Identifier <forward_incdec_operand> <num_literal>
212	<forward_incdec_operand>	→	++
213	<forward_incdec_operand>	→	--
214	<loop_stmts>	→	<statement>
215	<loop_stmts>	→	<loop_dribble>
216	<loop_dribble>	→	Dribble (<condition>) { <loop_stmts> <next_loopstmts> } <loop_pass> <loop_shoot>
217	<loop_pass>	→	<next_loop_pass> <loop_pass>

218	<loop_pass>	→	λ
219	<next_loop_pass>	→	Pass (<condition>) { <loop_stmts> <next_loopstmts> }
220	<loop_shoot>	→	Shoot { <loop_stmts> <next_loopstmts> }
221	<loop_shoot>	→	λ
222	<next_loopstmts>	→	<loop_stmts> <next_loopstmts>
223	<next_loopstmts>	→	λ
224	<next_loopstmts>	→	Timeout;
225	<next_loopstmts>	→	Resume;
226	<play_statement>	→	Play (<condition>) { <loop_stmts> <next_loopstmts><forward_incdec>}
227	<run_play_statement>	→	Run { <loop_stmts> <next_loopstmts><forward_incdec> } Play (<condition>);
228	<func_call>	→	identifier(<arguments>);
229	<arguments>	→	<argument> <arguments_end>
230	<arguments_end>	→	,<arguments>
231	<argument>	→	<literal>
232	<argument>		Identifier <array_id>
233	<array_id>		λ
234	<array_id>		[<element_index>]
235	<argument>	→	<type_casted>